



Swiftdigital

**Swiftdigital Web & API
VAPT 2026**

May 10, 2026

8

Table of Contents

Confidentiality and Distribution Restrictions

The conclusions and recommendations in this report represent the opinions of Securify.

Determinations of appropriate corrective action(s) are the responsibility of the entity receiving the report.

This report and/or any other materials furnished by Securify in connection with this engagement is confidential and may not be duplicated, modified, or otherwise reproduced and distributed without the express prior written consent of Securify or Swiftdigital. Because this work may contain copyrighted images or other material, permission from the copyright holder may also be necessary if you wish to reproduce.

Introduction

As part of an ongoing security program, **Swiftdigital** identified the need to conduct an application security assessment of its Web application & APIs.

This report presents the agreed scope, methodology, risk measurement model, summarized findings, and detailed technical observations for the selected assessment window.

The following lists the objectives of this assessment:

- Determine the overall security posture of the application
- Provide a list of key findings and recommendations for remediation
- Document the assessment scope, risk evaluation, and final outcomes
- Support remediation planning with actionable technical detail

This report includes the following parameters and results of the assessment:

- Securify's approach to the assessment
- Assessment scope
- Key findings listed with their qualitative risk assessment
- Detailed recommendations for each finding
- A remediation plan

Approach

Securify performed a Runtime Application Vulnerability Assessment of the Swiftdigital's web application, associated APIs, and all components defined within the assessment scope.

The assessment was conducted using industry-accepted methodologies, primarily based on the OWASP Web Security Testing Guide (WSTG) and OWASP ASVS, and involved both manual testing and automated analysis.

Runtime Application Vulnerability Assessment

The Runtime Application Vulnerability Assessment involved detecting security vulnerabilities through detailed examination and testing of the application in a runtime environment. This assessment emulates an attack by a skilled adversary in a controlled setting and allows Swiftdigital to ascertain the kinds of vulnerabilities that may be realistically exploited. A Runtime Application Vulnerability Assessment includes the following phases:

- ****Information Gathering**** – The application was reviewed as an anonymous, authenticated, and privileged user to understand differences in access and behavior. Technologies, frameworks, APIs, and third-party integrations were also identified to support targeted testing.
- ****Authentication Testing**** – Authentication mechanisms were evaluated to determine the strength of login controls, password policies, and account recovery processes. Protections against brute-force attempts and session takeover scenarios were also reviewed to ensure users are securely authenticated.
- ****Authorization Testing**** – Tests were conducted to identify weaknesses in access control, including attempts to access other users' data or privileged functionality.
- ****Session Management**** – Session handling was assessed to ensure secure creation, storage, and invalidation of session tokens.
- ****Input Validation Attacks**** – User-controlled input fields were tested with malformed and malicious data to identify injection flaws and logic bypasses. This included attempts to exploit unexpected behavior, access unprotected functionality, or inject vulnerabilities such as XSS, SQL Injection, and Command Injection.
- ****Business Logic Testing**** – The application workflows were reviewed to identify opportunities to misuse or bypass intended processes. This included testing for logic errors, insufficient validation, and scenarios where typical constraints could be circumvented.

Assessment Limitation

The ever-changing technology landscape and the increasing sophistication of attacks against networked systems are reasons why no entity can truthfully claim to identify all security issues or guarantee the lifetime security of an organization's network and applications. Note that this point-in-time assessment was based on a best-effort basis. It was also performed only in the environment provided by Swiftdigital. Thus, changes to the environment may impact the applicability of the results provided herein.

Securify cannot guarantee 100% coverage for any security assessment.

Findings and Recommendation

The sections below summarize the observed risks and provide the measurement criteria used to classify findings across the engagement.

Each finding is evaluated using the same impact and likelihood model so remediation can be prioritized consistently.

Risk Classification

The remainder of this report describes the vulnerabilities that Securify identified as part of the assessment, their impact, and recommendations for resolving the vulnerabilities. To assist in determining the risk posed by these vulnerabilities, Securify leverages the OWASP Application Security Risk Rating Methodology. The observations have been categorized based on technical Impact and Likelihood, explained below. These impact and likelihood scores may be further modified by Swiftdigital based on the business criticality of the target.

Measurement of Impact

Impact is an estimation of the potential damage via a successful exploit of a vulnerability. We'll use the following factors to help qualitatively determine the impact of a vulnerability.

- **Low Impact:** When most/all factors indicate limited consequences (e.g., non-sensitive data, no ability to alter/delete key data, and low victim count).
- **Medium Impact:** When about half of the factors suggest higher damage and half point to limited effects.
- **High Impact:** When most/all factors highlight significant damage (e.g., sensitive data loss, wide data corruption, and a large number of victims).

Impact	Description
Low	
Medium	
High	

Measurement of Likelihood

Likelihood is a qualitative estimation of the probability of an attacker exploiting the vulnerability in question. In order to determine the likelihood of exploitation, we can consider the following factors:

- **Low Likelihood:** Most/all factors suggest significant barriers to exploitation (e.g., complex skillset, limited attackers, high cost, or complex delivery).
- **Medium Likelihood:** About half the factors indicate ease of exploitation, while the other half show barriers.
- **High Likelihood:** Most/all factors point to easy and accessible exploitation (e.g., basic skills, low cost, and simple attack mechanisms).

Likelihood	Description
Low	
Medium	
High	

Overall Risk

- **Low Impact + Low Likelihood** = Low Risk
- **High Impact + High Likelihood** = Critical Risk

Zero-risk Issues

There are cases where a particular issue doesn't have direct security impact or likelihood of exploitation. These could be issues related to documentation or semantics that are suggested based on the security assessors' technical and industry experience. Such issues shall be reported with an Informational risk rating.



Vulnerabilities

Summary

Below is a summary of the vulnerabilities discovered during the assessment, ranked in order of risk as determined:

Vulnerability	Risk
Broken Authentication: Missing Session Validation for Zendesk JWT Generation	High
Broken Access Control Allows Unauthorized User Data Access	Medium

Detailed Vulnerabilities

Broken Authentication: Missing Session Validation for Zendesk JWT Generation

Risk: High

Description:

The application's Zendesk messaging JWT generation endpoint, located at ``https://swiftdigitalservices.com/ajax/zendesk_messaging_jwt.php``, is vulnerable to broken authentication. This endpoint accepts a POST request containing a base64-encoded user ID to generate a Zendesk JWT authentication token. However, the server fails to validate the user's session cookie, allowing an unauthenticated attacker to send requests without any session information and generate valid Zendesk JWT tokens for arbitrary user IDs. These tokens can then be used to access user-specific information and functionality within the Zendesk chat system.

Affected URL:

- https://swiftdigitalservices.com/ajax/zendesk_messaging_jwt.php

Impact and Likelihood:

- **Impact:** The impact of this vulnerability is high, as an attacker can generate Zendesk JWT tokens for any user. This allows for unauthorized access to user-specific data, including Personally Identifiable Information (PII), and the ability to perform actions such as initiating or participating in chats on behalf of other users. Such unauthorized access can lead to severe privacy breaches, impersonation, reputational damage, and potential regulatory non-compliance for the affected organization.
- **Likelihood:** The likelihood of exploitation is high because the vulnerability can be triggered by sending a simple POST request to the affected endpoint without requiring any prior authentication or valid session. An attacker can easily enumerate or guess user IDs, encode them, and generate valid Zendesk JWT tokens, making the attack straightforward and highly reproducible.

Steps to Reproduce:

Step 1: Identify a valid user ID within the application (e.g., by observing legitimate traffic or enumeration).

Step 2: Base64 encode the target user ID.

Step 3: Send a POST request to ``https://swiftdigitalservices.com/ajax/zendesk_messaging_jwt.php`` with the encoded user ID in the request body, ensuring no session cookies are included.

Step 4: Observe that the server responds with a valid Zendesk JWT authentication token for the specified user ID.

Step 5: Utilize the generated JWT token to access Zendesk functionalities as the impersonated user.

Recommendations:

- ****Authentication and Authorization:**** Implement robust authentication and authorization checks on the ``zendesk_messaging_jwt.php`` endpoint. Ensure that only authenticated and authorized users can request JWT tokens, and only for their own user ID.
- ****Session Validation:**** Validate the user's session cookie for all requests to the JWT generation endpoint. The server must verify that the request originates from a legitimate, authenticated user session.
- ****Access Control:**** Implement strict access control mechanisms to prevent unauthorized users from generating JWT tokens for other users. This may involve mapping the authenticated user's ID to the ID used in the JWT generation request.

- ****Secure JWT Generation:**** Review the JWT generation process to ensure it adheres to best practices, including using strong cryptographic algorithms, appropriate key management, and short expiration times for tokens.

References:

- https://owasp.org/www-project-top-ten/2021/A07_2021-Identification_and_Authentication_Failures

Broken Access Control Allows Unauthorized User Data Access

Risk: **Medium**

Description:

The application exhibits a broken access control vulnerability where a lower-privileged user can bypass intended authorization mechanisms. By directly accessing a specific PHP endpoint and manipulating URL parameters, an authenticated user can view details of other users, including sensitive information such as usernames and passwords, despite the user interface restricting such access. This allows for unauthorized data exposure and potential account compromise.

Affected URL:

- https://www.dev.blackrockgalleries.com/auction-admin/add_admin.php?id=137&brg=kkbiv65212

Impact and Likelihood:

- **Impact:** The potential impact of this vulnerability is significant, as it can lead to unauthorized access to other user accounts within the system. Exposure of usernames and passwords can result in account takeover, privilege escalation, and further compromise of the application or associated systems. This could lead to data breaches, reputational damage, and potential regulatory non-compliance.
- **Likelihood:** The vulnerability is highly likely to be exploited as it requires only an authenticated lower-privileged user to directly access a specific endpoint with a modified parameter. No complex tools or advanced techniques are necessary, making it easily discoverable and exploitable by an attacker with basic understanding of web application functionality.

Steps to Reproduce:

Step 1: Log in to the seller admin portal with a lower-privileged user account (e.g., 'manager' role).

Step 2: Navigate directly to the URL: `https://www.dev.blackrockgalleries.com/auction-admin/add\_admin.php?id=137&brg=kkbiv65212` (or any other valid user ID in the 'id' parameter).

Step 3: Observe that the page displays the details of the user corresponding to the 'id' parameter, including their username and password, despite the logged-in user's lower privileges.

Recommendations:

- ****Implement Server-Side Access Control:**** Ensure that all requests to sensitive resources are authorized on the server-side. Before displaying or processing user data, verify that the authenticated user has the necessary permissions to access that specific resource or user ID.
- ****Enforce Least Privilege:**** Design access control policies based on the principle of least privilege, ensuring users only have access to the data and functionality strictly required for their role.
- ****Validate Input Parameters:**** Implement robust input validation for all parameters, especially those used to identify resources (e.g., 'id'). While validation helps, it should always be complemented by server-side authorization checks.
- ****Avoid Direct Object References:**** Do not expose internal object references (like database IDs) directly in URLs or forms. Consider using opaque, non-sequential identifiers or mapping user-specific data without exposing underlying IDs.

References:

- https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control
- <https://cwe.mitre.org/data/definitions/284.html>

Appendix A

The appendix content below is static template material and will be rendered into reports that use this template.